You're almost there — sign up to start building in Notion today. Sign up or login

week-04-css-layout

Week 04 — CSS Layout

Session goal: by the end of class you can choose Flexbox or Grid, write mobile-first media queries, and use the five CSS positions.

Last class (Week 03) — recap

- **Basic CSS rules to style HTML (3 types)** — inline, internal (`<style>`), external stylesheet
- **Selectors** — tag, class, id, attribute, pseudo-class, pseudo-element
- **CSS specificity** — which rules win when more than one matches

This class (Week 04)

- **Flexbox** — one direction (a row *or* a column)
- **Grid** — rows *and* columns
- **Media queries** — responsive layouts that start small and grow with `min-width`
- **Position** — `static` , `relative` , `absolute` , `fixed` , `sticky`

Warm-up (8 min)

Flexbox lays items in **one direction** — a row *or* a column.

Grid lays items in **two directions** — rows *and* columns, like a table.

Paste this into `week-04/warmup.html` and open it with Live Server.

```
Flexbox (one line) Grid (rows + columns) A B C D E F A B C D E F
```

Example

Six identical boxes. Only the parent `display` changes.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>Flexbox
vs Grid</title> <style> body { font-family: system-ui, sans-serif; padding:
1rem; } h2 { font-size: 1rem; } .box { background: #ddd; padding: 1rem; tex
t-align: center; } .flex { display: flex; gap: 0.5rem; } .grid { display: g
rid; grid-template-columns: 1fr 1fr 1fr; gap: 0.5rem; } </style> </head> <b
ody> <h2>Flexbox — one row</h2> <section class="flex"> <div class="box">A</
div> <div class="box">B</div> <div class="box">C</div> <div class="box">D</
div> <div class="box">E</div> <div class="box">F</div> </section> <h2>Grid
— three columns, so two rows</h2> <section class="grid"> <div class="box">A
</div> <div class="box">B</div> <div class="box">C</div> <div class="box">D
</div> <div class="box">E</div> <div class="box">F</div> </section> </body>
</html>
```

You should see: Flexbox keeps all six in **one line** (they get skinny). Grid makes a **2 × 3 table** — D sits under A.

Task

1. Add a seventh box `G` to both sections. Flexbox: still one row. Grid: G starts a new row under A.
2. On `.flex`, add `flex-direction: column`. Flexbox becomes one **column**. Grid does not change.

Done when: you can say “Flexbox is 1D, Grid is 2D.”

Today we use both, plus mobile-first CSS, to fix layouts that break at 320px.

Flexbox vs Grid

Pick the tool that matches the problem.

Example

Reach for	When	Examples
Flexbox	One dimension (row **or** column)	Navbars, button groups, centring, card insides
Grid	Two dimensions (rows **and** columns)	Page shells, galleries, dashboards, forms

Class activity A — Flexbox gallery (8 min)

Paste this into `week-04/gallery.html`. You get six boxes in normal flow. **You** add the Flexbox.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>Flexbox
gallery</title> <style> body { font-family: system-ui, sans-serif; padding:
1rem; } .photo { background: #ddd; padding: 2rem 1rem; text-align: center;
} </style> </head> <body> <h1>Flexbox gallery</h1> <section class="galler
y"> <div class="photo">1</div> <div class="photo">2</div> <div class="phot
o">3</div> <div class="photo">4</div> <div class="photo">5</div> <div class
="photo">6</div> </section> </body> </html>
```

Add CSS so they sit in a wrapping row:

1. `display: flex` and `flex-wrap: wrap` on `.gallery`
2. `gap` on `.gallery` (not margins on `.photo`)
3. `flex: 1 1 160px` on `.photo` so items prefer 160px, then wrap

Done when: shrinking the window goes 4 → 2 → 1 across, with **no** horizontal scrollbar.

Flexbox

`display: flex` creates a **main axis** (`justify-content`) and a **cross axis** (`align-items`).

Example

Paste this file. Change **one** `justify-content` value at a time (`flex-start` , `center` , `flex-end` , `space-between` , `space-around` , `space-evenly`) and watch the three boxes move.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>Flexbox
axes</title> <style> * { box-sizing: border-box; } body { margin: 1.5rem; f
ont-family: system-ui, sans-serif; } .container { display: flex; flex-direc
tion: row; /* row | column | *-reverse */ flex-wrap: wrap; /* nowrap | wrap
*/ gap: 1rem; /* prefer gap over item margins */ justify-content: flex-star
t; /* flex-start | flex-end | center | space-between | space-around | space
-evenly */ align-items: stretch; /* stretch | flex-start | flex-end | cente
r | baseline */ min-height: 160px; border: 2px dashed #999; padding: 0.5re
m; } .item { background: #ddd; padding: 1.5rem 2rem; text-align: center; }
</style> </head> <body> <p>Main axis = <code>justify-content</code>. Cross
axis = <code>align-items</code>.</p> <div class="container"> <div class="it
em">1</div> <div class="item">2</div> <div class="item">3</div> </div> </bo
dy> </html>
```

Cheat sheet for items (add later — do not paste all three `flex` lines onto the same rule):

```
.item { flex-grow: 0; flex-shrink: 1; flex-basis: auto; flex: 1; /* equal c
olumns */ flex: 0 0 250px; /* fixed sidebar */ align-self: center; }
```

Example Explained

```
flex-start |▬▬ | center | ▬▬ | flex-end | ▬▬| space-between |■ ■ ■ | spa
ce-around | ■ ■ ■ | space-evenly | ■ ■ ■ |
```

- If `flex-direction: column`, main and cross axes swap
- Always ask: which way does main run?
- Use `gap` on the container — stop spacing children with random margins

> **Important:** Use `gap` on the container. Stop spacing flex children with random margins.

Class activity B — `justify-content` on the gallery (10 min)

Stay in `week-04/gallery.html` from activity A. Do **not** start a new file.

`justify-content` distributes **leftover space** on the main axis. If photos still have `flex: 1 1 160px`, they grow and eat that space — you will see almost no difference. First set:

```
.photo { flex: 0 0 120px; /* fixed width, so leftover space remains */ }
```

Then on `.gallery` try **one value at a time**. Save, look, then change it:

`flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `space-evenly`

```
flex-start |■■■ | center | ■■■ | flex-end | ■■■| space-between |■ ■ ■ | spa
ce-around | ■ ■ ■ | space-evenly | ■ ■ ■ |
```

Done when: you can point at the screen and name which value is showing.

CSS Grid

Grid places items on rows and columns at the same time.

Example

Paste this file. Resize the window on the **auto-fit** gallery — column count changes with **no** media query.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>CSS Grid
— columns</title> <style> * { box-sizing: border-box; } body { margin: 1.5r
em; font-family: system-ui, sans-serif; } h2 { font-size: 1rem; margin: 1.5
rem 0 0.5rem; } .box { background: #ddd; padding: 1.5rem; text-align: cente
r; } .grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 1rem;
} /* Other column recipes to try on .grid: grid-template-columns: 200px 1fr
200px; grid-template-columns: repeat(3, 1fr); */ .gallery { display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); gap: 1rem; }
</style> </head> <body> <h1>CSS Grid</h1> <h2>Three equal columns (<code>1f
r 1fr 1fr</code>)</h2> <div class="grid"> <div class="box">1</div> <div cla
ss="box">2</div> <div class="box">3</div> </div> <h2>Responsive gallery — r
esize me (<code>auto-fit</code> + <code>minmax</code>)</h2> <div class="gal
lery"> <div class="box">A</div> <div class="box">B</div> <div class="box">C
</div> <div class="box">D</div> <div class="box">E</div> <div class="box">F
</div> </div> </body> </html>
```

Class activity C — Grid gallery (8 min)

Same idea as activity A: six image boxes, **you** add the layout. This time use **Grid**, not Flexbox.

Paste this into `week-04/grid-gallery.html`

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>Grid gal
lery</title> <style> body { font-family: system-ui, sans-serif; padding: 1r
em; } .photo { background: #ddd; padding: 2rem 1rem; text-align: center; }
</style> </head> <body> <h1>Grid gallery</h1> <section class="gallery"> <di
v class="photo">1</div> <div class="photo">2</div> <div class="photo">3</di
v> <div class="photo">4</div> <div class="photo">5</div> <div class="phot
o">6</div> </section> </body> </html>
```

On `.gallery` add:

```
display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 1rem;
```

No media query for the column count. `auto-fit` + `minmax` does the reflow.

Done when: widening/narrowing the window changes how many columns you see, with **no** horizontal scrollbar at 320px.

Responsive Design (Mobile-First)

Write the **small** screen as the base. Add complexity with `min-width`.

Example

Paste this file. Use DevTools device toolbar at **320px**, **640px**, and **1024px** — cards go 1 → 2 → 3 columns.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>Mobile-f
irst cards</title> <style> * { box-sizing: border-box; } body { margin: 1.5
rem; font-family: system-ui, sans-serif; } .box { background: #ddd; paddin
g: 1.5rem; text-align: center; } img, video, svg { max-width: 100%; height:
auto; display: block; } .hero-title { font-size: clamp(2rem, 6vw, 4rem); ma
rgin: 0 0 1rem; } .container { width: min(1200px, 100% - 2rem); margin-inli
ne: auto; } .cards { display: grid; grid-template-columns: 1fr; gap: 1rem;
} @media (min-width: 640px) { .cards { grid-template-columns: repeat(2, 1f
r); } } @media (min-width: 1024px) { .cards { grid-template-columns: repeat
(3, 1fr); } } @media (prefers-reduced-motion: reduce) { *, *::before, *::af
ter { animation-duration: 0.01ms !important; transition-duration: 0.01ms !i
mportant; } } </style> </head> <body> <div class="container"> <h1 class="he
ro-title">Fluid heading</h1> <p>Base layout is one column. Wider viewports
add columns with <code>min-width</code>.</p> <div class="cards"> <div class
="box">Card 1</div> <div class="box">Card 2</div> <div class="box">Card 3</
div> <div class="box">Card 4</div> <div class="box">Card 5</div> <div class
="box">Card 6</div> </div> </div> </body> </html>
```

Example Explained

Name	Width	Typical device
Small	640px	Large phone / small tablet
Medium	768px	Tablet portrait
Large	1024px	Landscape / small laptop
Extra large	1280px	Desktop

> **Tip:** Add breakpoints where **your** layout breaks, not from a device checklist.

Class activity D — mobile-first breakpoints (10 min)

Use your Grid gallery (`week-04/grid-gallery.html`) or paste the six boxes from activity C. Remove `auto-fit` / `minmax` for this activity — **you** write the queries.

Start **small**. The base has **no** media query:

```
.gallery { display: grid; grid-template-columns: 1fr; gap: 1rem; }
```

Then add **all four** `min-width` breakpoints. Each one must change something you can see (column count):

Breakpoint	Query	Columns
Small	`@media (min-width: 640px)`	2
Medium	`@media (min-width: 768px)`	3
Large	`@media (min-width: 1024px)`	4
Extra large	`@media (min-width: 1280px)`	4, and a larger `gap`

Use DevTools device toolbar (or resize) at **320**, **640**, **768**, **1024**, and **1280**.

Done when: every breakpoint in the table changes the layout, and **320px** has no horizontal scrollbar.

Position

There are **five** `position` values. Learn them in **three** groups.

Group	Values	One-line idea
1	`static`	Simple flow. Boxes sit in HTML order.
2	`relative` vs `absolute`	Relative **nudges** and keeps its gap. Absolute **leaves** the line and pins to a parent.
3	`fixed` vs `sticky`	Fixed pins to the **window** forever. Sticky pins only while its **parent** is on screen.

1. Static — simple flow

Default. Three boxes in a row. No `top` / `left`. Neighbours never overlap.

2. Relative vs Absolute

Relative — the box moves, but its old seat stays empty (dotted ghost). The other boxes do not slide in.

Absolute — the box is taken out of the row. Neighbours close the gap. It pins to the nearest parent with `position: relative` (the red dot = that parent's top-left). If no such parent, it pins to the page.

3. Fixed vs Sticky

Fixed — pins to the **browser window**. Scroll the page: it does not move. It ignores its parent.

Sticky — behaves like relative, then sticks (like fixed) when you hit `top: 0`. It only sticks **inside its parent**. When the parent scrolls off the screen, sticky goes with it.

Example

Paste into `week-04/position.html`. Look at the boxes first, then **scroll** for fixed vs sticky.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1"/> <title>CSS posi
tion — boxes</title> <style> * { box-sizing: border-box; } body { margin:
0; font-family: system-ui, sans-serif; line-height: 1.45; color: #1a237e; }
h1 { font-size: 1.4rem; margin: 1.5rem 1.5rem 0.25rem; } h2 { font-size: 1.
05rem; margin: 1.75rem 1.5rem 0.35rem; } .lead { margin: 0 1.5rem 1rem; } .
sq { width: 72px; height: 72px; border-radius: 6px; } .purple { background:
#7c4dff; } .navy { background: #1a237e; } .mint { background: #69f0ae; } .p
ink { background: #f8bbd0; } .window { margin: 0 1.5rem 1rem; padding: 10p
x; background: #1a237e; border-radius: 10px; } .window-bar { height: 10px;
margin-bottom: 8px; background: #90caf9; opacity: 0.35; border-radius: 4px;
} .window-body { background: #3949ab; min-height: 160px; border-radius: 6p
x; padding: 12px; } .row { display: flex; gap: 10px; align-items: flex-star
t; } /* relative: ghost = old seat; box is nudged */ .slot { position: rela
tive; width: 72px; height: 72px; flex: 0 0 72px; } .ghost { position: absol
ute; inset: 0; border: 2px dotted #fff; border-radius: 6px; } .nudge { posi
tion: relative; top: 22px; left: 22px; } /* absolute: parent is the frame;
red dot = origin */ .abs-frame { position: relative; min-height: 170px; } .
origin { position: absolute; top: 8px; left: 8px; width: 12px; height: 12p
x; background: #ff1744; border-radius: 50%; z-index: 2; } .abs-child { posi
tion: absolute; top: 28px; left: 28px; } /* fixed: pins to the window */ .f
ixed-sq { position: fixed; right: 1rem; bottom: 1rem; z-index: 40; box-shad
ow: 0 4px 16px rgb(0 0 0 / 0.35); } /* sticky: pins only while the pink par
ent is on screen */ .sticky-parent { margin: 0 1.5rem 2rem; min-height: 160
vh; background: #f8bbd0; border-radius: 10px; padding: 8px; } .sticky-sq {
position: sticky; top: 8px; } </style> </head> <body> <h1>Five positions, t
hree ideas</h1> <p class="lead">Coloured squares. Then scroll to the pink p
anel.</p> <h2>1. Static — simple flow</h2> <p class="lead">HTML order. Noth
ing is offset.</p> <div class="window"> <div class="window-bar"></div> <div
class="window-body"> <div class="row"> <div class="sq purple"></div> <div c
lass="sq navy"></div> <div class="sq mint"></div> </div> </div> </div> <h2>
2. Relative vs Absolute</h2> <p class="lead"><strong>Relative</strong> — mi
ddle box moves. Dotted ghost = the seat it still owns. Neighbours stay put.
</p> <div class="window"> <div class="window-bar"></div> <div class="window
-body"> <div class="row"> <div class="sq purple"></div> <div class="slot">
<div class="ghost"></div> <div class="sq navy nudge"></div> </div> <div cla
ss="sq mint"></div> </div> </div> </div> <p class="lead"><strong>Absolute</
strong> — mint box leaves the row. Neighbours close the gap. It is measured
from the <strong>red dot</strong> (parent's top-left). Parent has <code>pos
ition: relative</code>.</p> <div class="window"> <div class="window-bar"></
div> <div class="window-body abs-frame"> <span class="origin" title="parent
origin"></span> <div class="row"> <div class="sq purple"></div> <div class
```

```
="sq pink"></div> </div> <div class="sq mint abs-child"></div> </div> </div
> <h2>3. Fixed vs Sticky — scroll now</h2> <p class="lead">The mint square
at the bottom-right is <strong>fixed</strong>. It stays on the window the w
hole time.</p> <div class="sq mint fixed-sq" title="fixed"></div> <p class
="lead">The mint square in the pink panel is <strong>sticky</strong>. It st
icks to the top while pink is on screen, then scrolls away with pink.</p> <
div class="sticky-parent"> <div class="sq mint sticky-sq"></div> <p style
="padding: 1rem;">Keep scrolling inside this pink parent…</p> <p style="pad
ding: 1rem;">Sticky is still here, stuck to the top.</p> <p style="padding:
1rem;">When this pink box leaves the window, sticky leaves too. Fixed would
not.</p> </div> </body> </html>
```

What you should see:

- **Static** — three squares in a straight row.
- **Relative** — middle square shifted; white dotted ghost stays; gap is not filled.
- **Absolute** — only two squares in the row; mint square sits off the **red dot**, not in the line.
- **Fixed** — mint square glued to the bottom-right of the **window** while you scroll.
- **Sticky** — mint square sticks at the top of the **pink parent**, then disappears when pink is gone.

Class activity E — three comparisons (8 min)

Open `position.html`:

1. Static: count the boxes. Are they in HTML order with even gaps?
2. Relative vs absolute: which one leaves a ghost? Which one lets neighbours close the gap? What is the red dot?
3. Scroll slowly. Point at **fixed** (window) vs **sticky** (pink parent). When pink leaves, which square is still on screen?

Done when: you can finish these three sentences:

- Static is …
- Relative keeps a gap; absolute …
- Fixed follows the window; sticky follows …

Learning resources

Practice these after class. Bookmark them — you will reuse Flexbox and Grid all term.

Resource	What it is	Link
Flexbox Froggy	Game: 24 levels of `justify-content`, `align-items`, `flex-direction`	flexboxfroggy.com
Grid Garden	Game: 28 levels of `grid-column`, `grid-row`, `grid-template`	cssgridgarden.com
Bootstrap	Component library (grid, navbar, cards). Read the layout docs after you can write Flexbox and Grid yourself	getbootstrap.com · Layout

Do Flexbox Froggy before Grid Garden. Open Bootstrap when you want a ready-made grid — not as a substitute for today's CSS.

Homework

Due before Week 05. Submit in the class channel / LMS as listed for each task.

Task 1 — Flexbox Froggy + Grid Garden

Complete Flexbox Froggy (all 24) and Grid Garden (all 28).

Submit: screenshots of both completion screens.

Task 2 — Grey-box layout clone

Recreate a real homepage **structure only** (grey boxes, no real images/brand copy). Use **Grid** for the page shell and **Flexbox** inside header / cards.

Submit: HTML + CSS + a screenshot next to the original (or a link to the original).

Task 3 — Make the profile responsive

Take your Week 02–03 profile and make it fully usable at 320px: nav, card, form, no horizontal scroll. Mobile-first `min-width` queries where needed.